Сервис расписания

Развёртывание и сопровождение

Руководство для студентов и сопровождающих

Документ 1.1 • 10 ноября 2026 • Приложение 0.14.0

Содержание

1. Как пользоваться руководством . . . 3
2. Паспорт и подтверждённое состояние . . . 3
3. Устройство системы и сохранность данных . . . 4
4. Доступ, ресурсы и сеть . . . 5
5. Установка Docker на Ubuntu . . . 5
6. Получение именно нужного кода . . . 6
7. Настройки и секреты . . . 7
8. Сборка и первый запуск . . . 7
9. Домен, HTTPS и внешняя сеть . . . 8
10. Администратор, диспетчер и преподаватели . . . 8
11. Почта и вход по коду . . . 9
12. Первый семестр и импорт данных . . . 9
13. Расчёт, переносы и утверждение . . . 10
14. Локальная проверка на Windows . . . 10
15. Автоматические проверки в контейнерах . . . 11
16. Повседневное управление . . . 11
17. Резервное копирование . . . 12
18. Плановые копии и контроль места . . . 13
19. Восстановление и проверка копии . . . 13
20. Обновление и откат . . . 14
21. Диагностика по симптомам . . . 15
22. Приёмка и передача следующему сопровождающему . . . 16
23. Шпаргалка путей и команд . . . 17
24. Источники и границы документа . . . 18

Руководство для студентов, диспетчера и следующего сопровождающего. Версия документа **1.1**, 10 ноября 2026 года. Базовая версия приложения — **0.14.0**, ветка `work`.

Репозиторий: https://github.com/alexander1221/schedule-service. Основной путь развёртывания: Ubuntu Server 24.04 LTS, Linux x86_64, Docker Engine и Compose, PostgreSQL 16, Caddy. Отдельно описана проверка на Windows с Docker Desktop. Python и Node.js для приложения устанавливаются внутри образов. Python 3.11+ на сервере нужен вспомогательному скрипту резервного копирования.

1. Как пользоваться руководством

Существующий сервер начинайте с паспорта и диагностики, а не с установки. Не создавайте заново `.env`, не меняйте имя Compose-проекта, не очищайте тома. Сначала установите, какие контейнеры и данные уже используются. Если не знаете пароль или назначение сервера, получите сведения у владельца.

На новом сервере выполняйте разделы 4–11 последовательно, затем контрольный сценарий. Команды для Linux рассчитаны на Bash под `root` или на администратора с согласованным `sudo`. В PowerShell используйте только блоки с соответствующей подписью. При ошибке остановитесь: успешность следующей команды не исправляет предыдущую.

В PDF и Word длинные строки могут переноситься визуально. Команды удобнее копировать из Markdown. В Linux знак `\` продолжает команду; в PowerShell для этого используется обратный апостроф. Заглушки `raspisanie.example.org`, `SERVER_IP` и имена архивов заменяются фактическими значениями. Это не адреса работающего сервиса.

Маршруты чтения:

- Новый сервер: разделы 2–13, затем 17–19 и 22.
- Проверка студентом на Windows: разделы 2–3, 14–15 и 22.
- Уже работающий сервер: разделы 2, 16–21 и 23.
- Передача следующему студенту: раздел 22 и заполненный паспорт.

Комплект содержит эту инструкцию в трёх форматах, скрипт копирования, отдельную конфигурацию тестов, контрольные суммы и описание. **Исходный код приложения в комплект не входит:** его получают из репозитория или Git bundle по разделу 6.

2. Паспорт и подтверждённое состояние

Заказчик выбрал сервер **185.11.247.226**, указанный адрес — https://185.11.247.226. Это план размещения, а не подтверждение, что на сервере уже установлено расписание. Домен/путь расписания, SSH-доступ и текущую конфигурацию требуется уточнить.

Поле	Что известно / что заполнить
Сервер, IP, площадка	Планируется 185.11.247.226; площадку и доступ уточнить с администратором
ОС, архитектура, ресурсы	Целевая ОС: Ubuntu 24.04, x86_64; фактические данные проверить
SSH-пользователь, порт, отпечаток	Заполнить; секреты хранить отдельно
Каталог нового сервера	В примерах `/opt/raspisanie`
Compose-проект	Для новой установки `raspisanie`; существующее имя не менять
Ветка разработки	`work`; наличие на GitHub проверить
Последний известный коммит исправлений	`7cd4838`; это ориентир, не универсальный SHA для отката
Версия приложения	`0.14.0`; фактическую проверить в контейнере
URL, DNS, TLS	Заполнить после проверки

Поле	Что известно / что заполнить
База	PostgreSQL 16, том Compose `pgdata`
Копии и ответственные	Заполнить локальный путь, внешнее хранилище, срок хранения

В среде разработки прошли 106 тестов ядра, два медленных были пропущены; отдельно прошли 13 новых проверок Django на SQLite. Полный API/UI-прогон, ruff, mypy, PostgreSQL и Docker-развёртывание в том окружении не были подтверждены. Эта инструкция задаёт порядок их выполнения, **не объявляет их уже пройденными**.

В `0.14.0` исправлены снятие прежней роли, обход утверждения через админку и конфликтующие закрепления при повторном переносе. Миграция `0008` оставляет последний из дублей утверждённым, остальные сохраняет как черновики.

Открытые ограничения: дата начала действия нового варианта ещё не реализована; календарь требует проверки раннего окончания киноклуба и выбора текущего семестра; автоматические права на площадки и оборудование неполны. Новые правила объединения максимум двух групп, смен и окон также ещё не полностью внедрены. Перед реальными занятиями сверяйтесь с `docs/OPEN_QUESTIONS.md` и `docs/CODE_AUDIT_2026-11-10.md`. Не выдавайте отсутствие ошибок установки за соответствие всем требованиям учебного отдела.

Особенность выбранного сервера: возможный общий nginx

В переданном комплекте equipment-bot этот же IP описан как сервер с nginx на 80/443 и панелью оборудования за ним. Это исторические сведения из инструкции от 09.11.2026, а не свежая проверка: остановленный контейнер оборудования не означает свободные порты nginx.

На этом сервере не запускайте основной Compose расписания с Caddy до проверки портов. Сначала выполните только чтение состояния через согласованный SSH-доступ:

```
sudo ss -lntp
sudo docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
sudo systemctl status nginx --no-pager
```

Если nginx обслуживает другой проект, обычный запуск Caddy конфликтует с ним. Нельзя останавливать nginx или заменять его сайт только ради освобождения 80/443. Нужно выбрать отдельный домен и согласовать nginx → локальный порт приложения, либо иной отдельный адрес/порт. Основной Compose сейчас описывает самостоятельный Caddy; точный override для совместного размещения готовится после проверки действующей схемы.

Простая замена `SITE_ADDRESS` не освобождает host ports: они заданы в Compose. Размещение на `/schedule/` также не является готовой настройкой: корневые пути приложения, API и статики потребуют отдельной проверки. Разделы установки с нуля ниже до согласования схемы используйте как описание отдельного стенда, а не как команды переключения этого сервера.

3. Устройство системы и сохранность данных

Путь запроса: браузер → Caddy:80/443 → `web`:8000 → PostgreSQL. Отдельный `worker` читает очередь из той же базы и запускает решатель OR-Tools. Интерфейс React собирается во время сборки образа, а не на компьютере диспетчера.

Сервис	Назначение	Что проверять
`db`	PostgreSQL: справочники, нагрузка, варианты, пользователи, очередь	`healthy`, подключения и резервные копии
`web`	Django/Gunicorn: админка, API, портал	HTTP-ответ, вход, миграции

Сервис	Назначение	Что проверять
`worker`	Фоновый расчёт расписаний	Очередь движется, результаты сохраняются
`caddy`	Внешний доступ, TLS, статические файлы	Адрес, сертификат, интерфейс и стили

Тома: `pgdata` — база; `static` — собранная статика; `caddy_data` — TLS-ключи и состояние Caddy; `caddy_config` — служебное состояние. Имена обычно начинаются с имени проекта, например `raspisanie_pgdata`; проверяйте через `docker volume ls` и `docker inspect`.

Код находится в образе. Изменение файлов на сервере не меняет уже работающий контейнер: нужна сборка. Данные в именованных томах переживают замену контейнеров. Но `docker compose down -v` удаляет тома: для обычной остановки эта команда не применяется. Образ, Git и копия папки проекта не заменяют дамп PostgreSQL.

У варианта расписания хранится снимок задачи. Это позволяет читать старый вариант после изменения справочников. Утверждение делает его видимым преподавателям, но пока не задаёт дату перехода внутри семестра. Решение о публикации принимает диспетчер, а не ИИ-агент.

4. Доступ, ресурсы и сеть

Получите доступ к серверу, репозиторию, DNS, сетевому экрану, почте и закрытому хранилищу копий. Определите владельца сервиса и человека, который разрешает простой и восстановление. Реальные ФИО, почта, нагрузка и архивы базы не помещаются в публичный Git или в учебные примеры.

Ориентир для пилота: 4 vCPU, 8 ГБ RAM и 20 ГБ диска, с последующей проверкой на реальной нагрузке. Это не гарантия времени расчёта. При нехватке памяти уменьшайте число потоков решателя; отдельно учитывайте место под образы, журналы и резервные копии. ARM и иные платформы этим комплектом не проверены.

На компьютере сопровождающего:

```
ssh -p 22 USER@SERVER_IP
```

Сверьте SSH-отпечаток с администратором. Дальнейшие Linux-команды выполняются на сервере:

```
cat /etc/os-release
uname -m
free -h
df -h /
timedatectl
ss -lntp
```

Для установки нужны исходящие HTTPS-соединения к GitHub, реестрам Docker, PyPI и npm, включая адреса скачивания из сообщений сборки. Примеры проверки:

```
getent ahostsv4 github.com
curl -I --connect-timeout 15 --max-time 30 https://github.com
curl -I --connect-timeout 15 --max-time 30 https://pypi.org
curl -I --connect-timeout 15 --max-time 30 https://registry.npmjs.org
```

DNS или ping не подтверждают доступ к HTTPS. Ответ главной страницы PyPI не доказывает доступ ко всем колёсам. Для реальных писем нужен SMTP выбранной организации. Прокси и зеркала согласует администратор; отключать TLS или проверку хэшей для обхода ошибки не нужно.

5. Установка Docker на Ubuntu

Сначала выполните `docker version` и `docker compose version`. Если обе команды работают, переустановка не нужна. Для уже настроенного сервера сохраните текущий источник пакетов. Не смешивайте пакеты Ubuntu `docker.io` с Docker CE без отдельного плана перехода.

Для **новой машины без Docker** ниже приведён вариант официального репозитория Docker. Перед выполнением сверьте его с инструкцией Docker для Ubuntu.

```
apt-get update
apt-get install -y ca-certificates curl git python3 openssl
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
chmod a+r /etc/apt/keyrings/docker.asc
```

Только после успешного скачивания ключа:

```
cat > /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
apt-get update
apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
systemctl enable --now docker
```

Контроль:

```
docker version
docker compose version
docker run --rm hello-world
```

Ошибка скачивания `hello-world` может означать проблему реестра. Не запускайте случайные установочные скрипты из интернета ради её устранения. Учитывайте, что доступ к Docker позволяет управлять контейнерами и ресурсами хоста; его выдают по правилам организации.

6. Получение именно нужного кода

На новом сервере, **если ветка `work` уже опубликована**:

```
git clone --branch work \
  https://github.com/alexander1221/schedule-service.git /opt/raspisanie
cd /opt/raspisanie
git status --short
git branch --show-current
git log -3 --oneline
git rev-parse HEAD
```

На дату подготовки комплектов коммиты `work` были локальными; нельзя считать, что сервер уже сможет скачать их с GitHub. Публикация выполняется владельцем отдельно. Для передачи без публикации можно создать Git bundle.

На Windows, из локального репозитория:

```
git bundle create ..\schedule-service-work.bundle work
git bundle verify ..\schedule-service-work.bundle
scp -P 22 ..\schedule-service-work.bundle USER@SERVER_IP:/tmp/
```

На **новом сервере**, где `/opt/raspisanie` ещё не существует:

```
git clone --branch work /tmp/schedule-service-work.bundle /opt/raspisanie
cd /opt/raspisanie
git rev-parse HEAD
```

У такой копии `origin` указывает на bundle, а не GitHub. Переход на сетевой remote согласуйте отдельно; не выполняйте сетевой `pull`, предполагая, что он уже настроен. Передачу через `/tmp` завершайте переносом bundle в контролируемое хранилище. Для существующего репозитория сначала изучите состояние, не клонируйте поверх него.

Файлы комплекта должны находиться в `docs/deployment/` внутри репозитория. Если комплект передан отдельно от кода, скопируйте его туда: относительные пути тестовой конфигурации рассчитаны именно на это расположение. Сверьте версию кода с руководством. Запишите полный SHA в паспорт; короткого номера версии недостаточно.

7. Настройки и секреты

На новой установке:

```
cd /opt/raspisanie
test -e .env || cp .env.example .env
chmod 600 .env
nano .env
```

Проверка `test` сохраняет существующий файл. Для генерации отдельных строк пароля и ключа дважды выполните `openssl rand -hex 32`. Впишите результат через редактор, без публикации в чате. Использование шестнадцатеричных строк избегает неоднозначной интерполяции `$` в Compose.

Общие параметры новой установки:

```
COMPOSE_PROJECT_NAME=raspisanie
DB_NAME=schedule
DB_USER=schedule
DB_PASSWORD=REPLACE_WITH_RANDOM_DATABASE_PASSWORD
DJANGO_SECRET_KEY=REPLACE_WITH_ANOTHER_RANDOM_SECRET
SOLVER_MAX_WORKERS=3
```

Обе заглушки обязательно заменить. `COMPOSE_PROJECT_NAME` привязывает последующие команды к одним томам. На существующем сервере сначала определите фактическое имя через `docker compose ls` и метки контейнеров. Новый префикс может создать пустую базу рядом со старой, что выглядит как потеря данных.

Пилот по HTTP — только localhost или согласованная закрытая сеть:

```
SITE_ADDRESS=:80
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
DJANGO_CSRF_TRUSTED_ORIGINS=http://localhost,http://127.0.0.1
DJANGO_SECURE_COOKIES=0
```

При обращении по IP добавьте фактический IP в оба списка, в список origins — с `http://`. Доступ извне к реальным учётным данным организуйте через HTTPS.

Рабочий домен с HTTPS, после подготовки DNS:

```
SITE_ADDRESS=raspisanie.example.org
DJANGO_ALLOWED_HOSTS=raspisanie.example.org
DJANGO_CSRF_TRUSTED_ORIGINS=https://raspisanie.example.org
DJANGO_SECURE_COOKIES=1
```

Замените домен во всех трёх строках. `ALLOWED_HOSTS` содержит имена без протокола; origins — адреса с протоколом. Не ставьте `*` ради устранения ошибки. `web` и `worker` принудительно используют PostgreSQL и `DJANGO_DEBUG=0` из Compose; не переносите настройки разработчика на рабочий сервер.

Переменные текущей оболочки могут влиять на подстановку Compose. Если ранее запускали локальные проверки, откройте чистую SSH-сессию/PowerShell и проверьте отсутствие случайных `DB_*` или `COMPOSE_*`. Порядок приоритетов описан в Docker Docs. Команда `docker compose config` раскрывает секреты: для обычной проверки используйте `docker compose config -q`.

8. Сборка и первый запуск

На сервере из `/opt/raspisanie`:

```
docker compose config -q
docker compose build
docker compose up -d db
docker compose ps
```

Ожидается успешная сборка и `healthy` у базы. Dockerfile устанавливает Python-зависимости по lock-файлу, собирает React в отдельном этапе Node и запускает приложение обычным пользователем контейнера. Отдельные `setup.sh`, npm и Python-окружение на хосте для этого не нужны.

До запуска обработчика выполните одноразовый проход инициализации:

```
docker compose run --rm --no-deps web python manage.py check
```

Entry point контейнера ожидает PostgreSQL, применяет миграции, создаёт роли и собирает статику; затем запускается `check`. **Продолжайте только после кода выхода 0.** Этот порядок не позволяет `worker` начать работу раньше создания таблиц.

```
docker compose up -d
docker compose ps -a
docker compose logs --tail=100 web worker caddy
docker compose exec web python -c \
  'from scheduler import __version__; print(__version__)'
```

Все четыре сервиса должны работать, база — быть здоровой. Up у Gunicorn сам по себе не проверяет доступность приложения. Для HTTP-пилота:

```
curl -sS -o /dev/null -w 'login: %{http_code}\n' \
  http://127.0.0.1/login/
curl -sS -o /dev/null -w 'app: %{http_code}\n' \
  http://127.0.0.1/app/
```

Ожидаются `200` на странице входа и перенаправление `302` для неавторизованного доступа к интерфейсу. После входа `/app/` должен загрузить сетку и стили. Для домена используйте HTTPS и имя из `.env`, а не случайный IP.

9. Домен, HTTPS и внешняя сеть

В этом проекте роль внешнего прокси выполняет **Caddy в Compose**. Дополнительно устанавливать nginx и Certbot по образцу другого проекта не требуется. Если 80/443 уже заняты, согласуйте размещение с администратором; не останавливайте чужой сайт. Существующий общий прокси требует отдельной схемы, не описанной командами установки с нуля.

Для домена проверьте запись A, при наличии AAAA — рабочий IPv6, маршрутизацию и входящие TCP 80/443. Для выбранного SSH-порта также должно быть разрешение. PostgreSQL и Gunicorn наружу не публикуются: в основном Compose нет host ports для 5432 и 8000.

```
getent ahosts raspisanie.example.org
ss -lntp
docker compose logs --tail=100 caddy
docker compose exec caddy caddy validate \
  --config /etc/caddy/Caddyfile --adapter caddyfile
curl -I https://raspisanie.example.org/login/
```

Проверьте сайт из внешнего браузера: сертификат должен быть доверенным и соответствовать имени. Нельзя считать `curl -k` доказательством корректного TLS. Caddy управляет получением и продлением сертификатов; сохранность его состояния зависит от тома `caddy_data`. Подробности — официальная документация Caddy.

Проверьте `ufw status verbose` и правила хостинга. Перед включением/изменением firewall убедитесь, что не потеряете SSH и доступна консоль восстановления. Публикуемые Docker порты требуют отдельного внимания: обычные правила UFW не всегда ограничивают их так, как ожидается. Не публикуйте базу ради удобства проверки.

10. Администратор, диспетчер и преподаватели

Создайте первого администратора интерактивно:

```
docker compose exec web python manage.py createsuperuser
```

Введите почту и пароль, откройте `/admin/`. Общий пароль не включают в инструкцию. Суперпользователь — аварийная/административная учётная запись, не повседневная роль студента.

Создайте отдельного диспетчера: запись пользователя, разрешённый адрес в белом списке, роль «Диспетчер». У существующего пользователя смена роли через белый список синхронизирует управляемые группы и `is_staff`. Для обычного преподавателя доступ к админке и API должен исчезнуть. Проверяйте на отдельной учётке: суперпользователь намеренно сохраняет полномочия.

Преподавателю указывают почту в карточке и выполняют действие «Разрешить вход по почте». Он входит на `/login/` по коду и видит `/me/`. Если почта есть только у пользователя, но не

связана с карточкой преподавателя, личная страница может сообщить об отсутствии привязки.

Проверьте роли отдельными сеансами браузера: диспетчер меняет расписание, преподаватель — только свои пожелания до срока. При нехватке прав на оборудование или площадки администратор проверяет разрешения: известное ограничение `init_roles` не следует обходить выдачей суперпользователя всем.

11. Почта и вход по коду

Без `EMAIL_HOST` используется консольная почта: письмо не отправляется, код виден в журнале `web`. Это допустимо для изолированного учебного стенда, но не является настроенной доставкой писем. Журналы такого стенда содержат коды; не публикуйте их целиком.

Параметры реальной почты, выданные ответственным за почтовый сервис:

```
EMAIL_HOST=smtp.example.org
EMAIL_PORT=465
EMAIL_HOST_USER=raspisanie@example.org
EMAIL_HOST_PASSWORD=REPLACE_WITH_MAIL_APPLICATION_PASSWORD
EMAIL_USE_SSL=1
EMAIL_USE_TLS=0
DEFAULT_FROM_EMAIL=raspisanie@example.org
```

Адреса здесь — заглушки. Для STARTTLS обычно выбираются порт 587, SSL=0, TLS=1; точные значения задаёт почтовая служба. Не включайте оба режима. Пароль приложения отличается от основного пароля ящика.

После изменения `.env` пересоздайте сервисы и проверьте доставку:

```
docker compose up -d --force-recreate web worker
docker compose exec web python manage.py test_email YOUR_REAL_EMAIL
```

Успех команды дополняется реальным получением письма, проверкой спама и входом по коду. Код действует 10 минут, даётся пять попыток; повторная отправка ограничена. Не запрашивайте десятки кодов подряд. Вход незнакомого адреса не должен раскрывать наличие его в белом списке.

12. Первый семестр и импорт данных

Учебную демонстрацию выполняйте на отдельной базе. Для неё можно создать вымышленные данные:

```
docker compose exec web python manage.py seed_synthetic \
  --seed 5 --groups 4 --teachers 10 --name "Учебная проверка"
```

Повторное создание с тем же названием отклоняется. Не удаляйте рабочие семестры ради повторения примера и не смешивайте синтетику с реальной нагрузкой.

Рабочий процесс диспетчера:

1. Создать семестр: название, начало, окончание занятий, чётность первой недели, суббота и срок пожеланий. Курсы и сетка пар создаются автоматически. Старые семестры не меняют свою сетку при обновлении значений по умолчанию.
2. Заполнить преподавателей, группы, аудитории, площадки и оборудование. Время закрытия площадки ограничивает доступные пары. Численность и вместимость учитываются только при включённой настройке.
3. Получить шаблоны справочников и нагрузки из админки. При переносе из старого расписания сначала сформировать черновики, прочитать лист «Что проверить», затем исправить нераспознанные записи.
4. Импортировать сначала с «Только проверить». Ошибки исправлять по листу, строке и колонке. ID существующей записи сохранять: это обновление, а не создание дубликата. Импорт при ошибках не должен частично менять данные.
5. Проверить синхронные блоки английского, пары по нагрузке, ручные занятия и закрепления. Существование флажка в модели не гарантирует поддержку каждой новой договорён-

ности решателем.

Документы для деталей: `docs/GUIDE.md`, `docs/XLSX_FORMAT.md`, `docs/OPEN_QUESTIONS.md`. Актуальные часы берутся из нагрузки преподавателей, не из случайных расхождений старого файла расписания.

13. Расчёт, переносы и утверждение

Сначала открыть «Проверить» у семестра. Красные ошибки требуют исправления; зелёная предварительная проверка ещё не доказывает существование полного решения. Затем «Построить расписание»: для первого осмотра дать несколько минут на вариант, посмотреть штрафы и занятость ресурсов.

Расчёт идёт в `worker`, страница может быть закрыта. Один семестр одновременно имеет один активный запуск. «Остановить и сохранить найденное» запрашивает корректную остановку; найденный вариант проверяется и сохраняется. Принудительный restart контейнера не равен этой кнопке.

В `/app/` выберите семестр, вариант, группу/преподавателя/аудиторию и неделю. Перенос мышью или выбором клетки сначала создаёт черновик. Список нарушений пересчитывается независимо; по переносу нельзя обещать готовый результат до пересборки. Английский блок переносится целиком.

«Закрепить и пересобрать» в `0.14.0` одной транзакцией заменяет закрепления переносимых вхождений и ставит расчёт в очередь. Другие пары и незатронутые недели сохраняются. При конфликте активного запуска черновик остаётся для повторной попытки. Сравните новый вариант с исходным; проверьте, что решатель не сдвинул нежелательные занятия.

«Утвердить» запускает независимый валидатор. Обычное редактирование статуса в форме закрыто; в семестре допускается один утверждённый вариант. Результат виден преподавателям и доступен в выгрузках. Изменение с конкретной даты пока не реализовано — не обещайте автоматическое сохранение старого расписания по прошедшим неделям.

При необходимости консольного расчёта сначала определите ID семестра в админке, затем подставьте его вместо `SEMESTER_ID`:

```
docker compose exec worker python manage.py solve_semester \
  SEMESTER_ID --time 300 --variants 3 --workers 3
```

Не запускайте несколько расчётов одного семестра и не ослабляйте запреты преподавателей ради красивого результата без решения диспетчера.

14. Локальная проверка на Windows

Нужны Git, запущенный Docker Desktop с Linux containers и доступ к реестрам. Откройте новое окно PowerShell; перейдите в папку, где лежит `docker-compose.yml`:

```
git switch work
git status --short
docker version
docker compose version
```

Для учебного запуска используйте отдельную копию репозитория и `.env` из раздела 7, но с `COMPOSE_PROJECT_NAME=schedule-local`. Пилотные адреса — localhost, secure cookies=0. Не используйте `.env` рабочего сервера. Порты 80/443 должны быть свободны; одновременно запускать серверный и локальный Compose на этих портах нельзя.

```
docker compose config -q
docker compose build
docker compose up -d db
docker compose run --rm --no-deps web python manage.py check
docker compose up -d
docker compose exec web python manage.py createsuperuser
docker compose ps -a
```

Проверяйте `$LASTEXITCODE` после каждой команды: 0 означает успех команды, а не полноту

проверки приложения. Откройте `http://localhost/admin/` и `http://localhost/app/`, выполните сценарий раздела 22.

Если нужен локальный дамп, не пропускайте бинарный вывод через PowerShell 5.1 `>`:

```
New-Item -ItemType Directory -Force backups | Out-Null
docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Fc -f
↪  /tmp/local-check.dump'
$scheduleDbId = docker compose ps -q db
docker cp "${scheduleDbId}:/tmp/local-check.dump" .\backups\local-check.dump
```

Укажите новое имя файла, если предыдущий снимок нужно сохранить. Для остановки — `docker compose stop`, для возвращения созданных контейнеров — `docker compose start`. Python и Node.js на Windows не требуются этим командам.

15. Автоматические проверки в контейнерах

В комплекте есть `Dockerfile.check` и **отдельный** `compose.test.yml`. У него собственная база с тестовым паролем, нет внешних портов и подключения рабочего `.env`. Не заменяйте им основной Compose и не подмешивайте его через второй `-f`. Используйте имя проекта `schedule-check`, отличное от рабочего.

Из корня репозитория, Bash или PowerShell:

```
docker build -t schedule-check-app:work .
docker compose -p schedule-check -f docs/deployment/compose.test.yml build
docker compose -p schedule-check -f docs/deployment/compose.test.yml run --rm checks-sqlite
docker compose -p schedule-check -f docs/deployment/compose.test.yml run --rm checks-postgres
docker compose -p schedule-check -f docs/deployment/compose.test.yml run --rm ui
```

Сначала должна успешно собраться базовая `schedule-check-app:work`: тестовый Dockerfile наследуется от неё. Если комплект находится вне `docs/deployment`, сначала исправьте размещение по разделу 6, а не меняйте произвольно контекст сборки.

Первая проверка выполняет pip check, ruff, форматирование без изменения файлов, mypy, проверку миграций и полный быстрый pytest на SQLite. Вторая запускает все веб-тесты на PostgreSQL, включая конкурентное утверждение. Третья выполняет typecheck, Vitest и сборку интерфейса. Рабочий Dockerfile сам по себе не запускает pytest или Vitest.

XML-отчёты появляются в `output/tests-sqlite.xml` и `output/tests-postgres.xml`. Если линтер остановил цепочку раньше pytest, XML может отсутствовать. Пропуски медленных тестов и MCP без SDK ожидаемы; конкурентный тест должен выполниться на PostgreSQL. Ни одного `FAILED`/`ERROR` не оставляют без разбора.

Дополнительно для изменений решателя, генератора или валидатора:

```
docker compose -p schedule-check -f docs/deployment/compose.test.yml run --rm --entrypoint python
↪  checks-sqlite -m pytest -q tests --slow -k synthetic
```

Для изменений MCP прогоняются обе версии SDK по `.github/workflows/ci.yml`; отсутствие SDK не считается проверкой протокола. После автоматических тестов нужен браузерный сценарий: Vitest работает в jsdom, а не проверяет реальный TLS, cookies и маршрутизацию.

```
docker compose -p schedule-check -f docs/deployment/compose.test.yml down
```

Эта команда удаляет контейнеры тестов, сохраняя их том. Рабочий стек не затрагивается при правильных `-p` и `-f`. Полный Docker-прогон комплекта в среде автора не выполнен; отмечайте фактические результаты в акте проверки.

16. Повседневное управление

Все Linux-команды ниже — из `/opt/raspisanie` с сохранённым `.env` и прежним именем проекта:

```
docker compose ps -a
docker compose logs --tail=100 web worker caddy
docker compose logs -f worker
```

`Ctrl+C` при просмотре logs завершает просмотр, а не контейнер. Остановку планируйте вне расчёта: сначала запросить её в приложении и дождаться результата.

Действие	Команда / смысл
Остановить весь стек	`docker compose stop`
Включить существующие контейнеры	`docker compose start`
Создать отсутствующие контейнеры	`docker compose up -d`
Применить изменённый `.env`	`docker compose up -d --force-recreate web worker caddy`
Применить код	Собрать образы и выполнить порядок обновления из раздела 20
Перезапустить процесс	`docker compose restart web`; не применяет новое окружение

Для изменения учётных параметров PostgreSQL отдельный порядок: пересоздание приложения не меняет пароль роли в сохранённом томе. Не меняйте одновременно DB_NAME, DB_USER и DB_PASSWORD, пытаясь исправить ошибку входа.

Политика `unless-stopped` обычно возвращает контейнеры после перезагрузки хоста, если они не были остановлены вручную. После `stop` нужен явный `start`. Проверьте `systemctl is-enabled docker` и запуск после согласованной перезагрузки. Расчёт, убитый вместе с контейнером, может быть отмечен неуспешным после периода отсутствия отклика; это не повод напрямую менять очередь SQL-командами.

17. Резервное копирование

Копия нужна до обновления, восстановления, изменения доступа и регулярно в эксплуатации. Рабочая PostgreSQL копируется через `pg_dump`, не копированием живого каталога тома. Он создаёт согласованный снимок базы; это ещё не доказательство, что её удалось восстановить. См. PostgreSQL: pg_dump.

Штатный `deploy/backup.sh` делает только дамп. Для полного комплекта используйте прилагаемый `schedule-backup.py`: стандартная библиотека Python, без дополнительных pip-пакетов. Установить на сервере:

```
install -m 600 docs/deployment/schedule-backup.py \
  /usr/local/sbin/schedule-backup.py
python3 --version
python3 /usr/local/sbin/schedule-backup.py \
  --project-dir /opt/raspisanie --project raspisanie \
  --output-dir /var/backups/raspisanie --include-caddy-data
```

Для другого имени проекта обязательно замените `--project`. Скрипт обращается к конкретному стеку; он не ищет «первую подходящую» базу.

Состав ZIP: бинарный `database.dump`, оглавление pg_restore, `.env`, основной Compose, Dockerfile, entry point, Caddyfile, lock-файлы, итоговый Compose с подставленными параметрами, SHA Git, статус дерева и контрольные суммы. При флаге `--include-caddy-data` копируется `/data` Caddy с закрытыми TLS-ключами. Дополнительные override-файлы и внешние службы автоматически не сохраняются: добавьте их в регламент отдельно. Код приложения целиком не включён; сохраните доступный релиз/bundle для записанного SHA.

Перед запуском убедитесь, что настройки и код никто одновременно не редактирует. Дамп БД согласован, но база, файлы конфигурации и TLS не образуют единую транзакцию. При плановом обновлении снимайте копию в спокойный период. При изменённом Git-дереве SHA не описывает все правки: сохраните и разберите их отдельно.

Ожидается `BACKUP OK` и новый архив с UTC-датой и уникальным суффиксом. При ошибке готовый ZIP не появляется; `.part` не принимается за копию. Скрипт проверяет формат `PGDMP`,

чтение оглавления и целостность ZIP. Он не запускает приложение из снимка, не шифрует архив, не удаляет старые копии и не останавливает сервисы.

Архив содержит секреты и персональные данные. Права 600/700 — ограничение доступа на Linux, **не шифрование**. Храните копии вне репозитория и веб-каталогов; организуйте закрытый вынос на другой носитель. Пример с компьютера сопровождающего:

```
scp -P 22 USER@SERVER_IP:/var/backups/raspisanie/SELECTED_BACKUP.zip .
```

Файл на том же диске не защищает от потери сервера. Ответственный проверяет наличие внешней копии и пробное восстановление, а не только сообщение успешного cron.

18. Плановые копии и контроль места

После ручного успешного запуска и пробного восстановления настройте cron. Ubuntu 24.04 предоставляет Python 3.12, подходящий скрипту; на другой ОС проверьте минимум 3.11. Дополнительно нужны `cron` и `flock` из util-linux.

```
apt-get install -y cron util-linux
systemctl enable --now cron
timedatectl
```

На новой установке создайте файл через редактор:

```
nano /etc/cron.d/raspisanie-backup
```

Содержимое для ежедневной копии в 03:15 **по системному часовому поясу**:

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
15 3 * * * root /usr/bin/flock -n /run/raspisanie-backup.lock /usr/bin/python3
↪  /usr/local/sbin/schedule-backup.py --project-dir /opt/raspisanie --project raspisanie --output-dir
↪  /var/backups/raspisanie --include-caddy-data >> /var/log/raspisanie-backup.log 2>&1
```

Сохраните завершающую новую строку и задайте `chmod 644 /etc/cron.d/raspisanie-backup`. Если файл уже существует, изучите его перед изменением. Часовой пояс приложения Europe/Moscow не меняет расписание cron.

Проверка после первого планового запуска:

```
tail -n 30 /var/log/raspisanie-backup.log
ls -lt /var/backups/raspisanie
df -h /
docker system df
```

Скрипт не выполняет ротацию. Владелец задаёт срок хранения и безопасную очистку проверенных старых копий; отдельно настраиваются вынос и уведомления об ошибках. Журнал тоже нуждается в ротации. Не добавляйте слепой `rm` по маске или `docker volume prune`: на сервере могут быть другие приложения.

19. Восстановление и проверка копии

Восстановление заменяет данные выбранной базы. Сначала нужен отдельный учебный стенд, затем — разрешение владельца и окно простоя для рабочего. Запишите снимок текущего состояния: изменения после даты выбранной копии будут потеряны при возврате к ней.

Выберите доверенный архив, проверьте дату и распакуйте в новый закрытый каталог. Здесь пути — пример; замените имя ZIP и каталог для своего случая:

```
BACKUP=/var/backups/raspisanie/SELECTED_BACKUP.zip
RESTORE_DIR=/root/raspisanie-restore-check
python3 -m zipfile -l "$BACKUP"
mkdir -m 700 "$RESTORE_DIR" && \
  python3 -m zipfile -e "$BACKUP" "$RESTORE_DIR" && \
  (cd "$RESTORE_DIR" && sha256sum -c SHA256SUMS.txt) && \
  cat "$RESTORE_DIR/manifest.json"
```

Каталог должен быть новым и пустым, архив — полученным из доверенного источника. До продолжения все суммы должны совпасть. Сверьте Compose-проект, Git SHA, дату и наличие нужного дампа. Контрольная сумма не доказывает доверенность источника и не заменяет парольное шифрование.

Подготовьте соответствующую версию кода. При восстановлении на чистой машине сначала создаются `.env`, база и контейнеры по разделам 6–8; не включайте SMTP с реальными адресами на пробном стенде. `.env` из архива переносите осознанно: старые пароли и токены могут быть уже отозваны. Совместимость схемы с выбранным кодом определяют до запуска миграций.

Для рабочего восстановления остановите **оба писателя базы** и убедитесь в остановке:

```
cd /opt/raspisanie
docker compose stop web worker
docker compose ps -a
```

db остаётся работающей; от Caddy в этот период возможен 502. Встроенный `deploy/restore.sh` версии 0.14.0 останавливает только web и возобновляет его даже после ошибок. Для этого регламента он не используется.

Примените дамп одной транзакцией с остановкой при ошибке:

```
docker compose exec -T db sh -c \
  'exec pg_restore -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean --if-exists --no-owner
  ↪  --single-transaction --exit-on-error' \
  < "$RESTORE_DIR/database.dump"
```

Если команда не вернула 0, не запускайте сайт: разберите ошибку, сохраните журнал и проверьте состояние базы. Не считайте произвольные предупреждения восстановления безвредными. Ключи восстановления описаны в документации PostgreSQL 16.

После успешного восстановления и выбора правильной версии кода:

```
docker compose run --rm --no-deps web python manage.py check
docker compose up -d web worker
docker compose logs --tail=100 web worker
```

Одноразовый запуск применяет миграции выбранного кода. Для точного отката это должен быть код из плана отката, а не случайная более новая ветка. Проверьте вход, семестры, нагрузку, варианты, статус утверждения и новый расчёт. Активный на момент снимка запуск не является продолжающимся процессом; обработчик должен корректно определить его состояние.

TLS-состояние восстанавливают только при утрате Caddy, отдельно от БД. Остановите `caddy`, сохраните текущую `/data`, затем копируйте содержимое `caddy-data` в `/data` его контейнера через `docker cp` и запускайте Caddy. Не копируйте архив поверх живого TLS-хранилища. При рабочем домене и доступном центре сертификатов возможен перевыпуск, но он не заменяет согласованное восстановление конфигурации.

Старую копию и предыдущие образы сохраняйте до приёмки владельцем. Пробное восстановление документируют: где, когда, какой SHA, какой архив, какие данные и функции проверены.

20. Обновление и откат

Обновление проводится вне активного расчёта. Перед ним: копия базы и настроек, полный SHA, список образов, чтение CHANGELOG и результатов тестов. Для `0.14.0` отдельно учесть миграцию статусов утверждения; она не удаляет варианты, но меняет статус старых дублей.

```
cd /opt/raspisanie
git status --short
git rev-parse HEAD
docker compose images
docker compose ps -a
```

Если есть локальные изменения, сначала разобраться с ними. Не выполнять `reset --hard`, `git clean` или принудительный checkout для устранения затруднения. Сохраните прежний образ тегом и при необходимости экспортируйте `docker image save` в закрытое хранилище: повторная сборка старого SHA может потребовать недоступных реестров и дать другой базовый образ.

Для чистой копии с настроенным GitHub remote и опубликованной веткой:

```
git fetch origin
git log --oneline HEAD..origin/work
git switch work
git merge --ff-only origin/work
docker compose build
```

После передачи проекта в стабильную ветку замените work на согласованную ветку/релиз. При отказе fast-forward остановитесь; не применяйте force. Для репозитория из bundle используйте согласованный новый bundle, а не эти сетевые команды вслепую.

Сначала успешно соберите новую версию, пока старые контейнеры ещё работают. Затем дождитесь завершения расчётов, повторите копию при новых изменениях данных и выполните:

```
docker compose stop web worker
docker compose run --rm --no-deps web python manage.py check
docker compose up -d web worker
docker compose logs --tail=100 web worker
```

После миграций проверьте версию, HTTP/HTTPS, вход, статику, очередь и контрольный перенос. Изменённый файл Caddyfile нужно отдельно провалидировать и применить Caddy; смена .env требует пересоздания контейнера, а не restart. Статические файлы публикуются раньше нового web, поэтому возможен краткий период несовпадения клиента и сервера: обновляйте в окно простоя.

Для отката на чистом дереве выбирается **реально записанный** старый SHA, затем совместимый образ/сборка. Если изменена схема, одного возврата Git недостаточно. План может потребовать восстановления базы из предобновительной копии с потерей последующих изменений. Обратная миграция не гарантирует возвращения прежних значений данных. Не используйте 7cd4838 автоматически как «последнюю рабочую версию» на любом сервере.

21. Диагностика по симптомам

Docker не отвечает или недостаточно прав

На Windows запустить Docker Desktop и Linux containers. На Linux проверить systemctl status docker. Различайте выключенный daemon и отказ доступа к сокету. Не исправляйте его выдачей всем полного доступа к Docker или изменением прав на сокет.

Не скачиваются образы, pip или npm

Смотрите конкретный адрес, DNS, тайм-аут, TLS, ошибку хэша и свободный диск. Не заменяйте lock-файлы произвольными версиями. Для Dockerfile есть BASE_IMAGE и NODE_IMAGE; образы PostgreSQL/Caddy используют выбранный администратором реестр. Работоспособность одного URL не доказывает всей цепочки загрузок.

Контейнер перезапускается, entrypoint не найден

Смотрите docker compose logs --tail=150 web worker. На Windows проверьте окончания строк deploy/*.sh: shebang с CRLF может дать no such file or directory или bad interpreter. Сохраните shell-файлы с LF и пересоберите образы; старый контейнер от правки файла не изменится. Предпочтительно получать серверную копию через Git на Linux.

PostgreSQL healthy, но приложение не подключается

healthy проверяет доступность PostgreSQL, а не пароль приложения. Сверьте имя базы, пользователя и неизменность пароля после первого запуска тома. Не удаляйте pgdata для «сброса». DB_HOST=db работает внутри сети Compose; localhost в контейнере означает сам контейнер.

После запуска всё пусто

Сравните имя Compose-проекта, список томов и папку запуска с паспортом. Частая причина — новый пустой стек с другим префиксом. Не импортируйте данные заново, пока не найдена прежняя база. Остановите ошибочный стек без удаления томов и восстановите правильный маршрут к данным.

HTTP 400, CSRF 403, вход зациклен

400 — проверить `DJANGO_ALLOWED_HOSTS`; CSRF — origin с правильным протоколом и работу прокси. При HTTP-пилоте нужны secure cookies=0; при HTTPS — 1. Проверьте один и тот же адрес в браузере и настройках. После изменения `.env` пересоздайте контейнеры. Не выключайте CSRF-проверку.

502, не открывается сайт или интерфейс без стилей

Проверить `web`, `caddy`, их логи, свободные порты, DNS и сертификат. Для интерфейса проверить успешную сборку frontend, `collectstatic`, том `static` и запросы `/static/` в инструментах браузера. Сборка без запуска миграций/публикации статики не завершает установку. После осознанной остановки web 502 ожидаем.

Письмо не пришло

Проверить карточку преподавателя, белый список, журнал кодов, SMTP и реальный `test_email`. Консольный backend не отправляет письма. Не публиковать коды из логов. После нескольких попыток учесть временные ограничения отправки и ввода.

Расчёт стоит в очереди или остановился

Посмотреть `worker`, активный запуск и журнал прогресса. Не стартовать второй worker/CLI для обхода ограничения одного семестра. При `unknown` увеличить время или пересмотреть данные с диспетчером. При `infeasible` искать конфликт запретов, блоков, аудиторий и закреплений. Ошибка независимого валидатора у результата решателя — повод для исправления кода, а не для утверждения вручную.

База восстановлена с ошибками

Оставить web/worker остановленными, проверить версию PostgreSQL, архив и первую ошибку. Не выдавать частичное восстановление за успех и не повторять случайные SQL-команды. Работать с сохранённой предвосстановительной копией и согласованным планом.

Закончилось место

Проверить `df -h`, `docker system df`, копии и логи. Образы и кэш очищаются после инвентаризации, тома — только по отдельному решению. Не выполнять `docker system prune -a --volumes` на рабочем сервере. Ограничение размера логов требует отдельного регламента, в основном Compose оно не настроено.

Что передать разработчику

Дата, действие, полный SHA, версия, ОС/архитектура, команда, код выхода, очищенный фрагмент ошибки, `docker compose ps -a`, XML тестов и воспроизведение на вымышленных данных. Не отправлять `.env`, `docker compose config`, резервные копии, токены, полные персональные данные и журналы кодов входа. Для реальных данных используйте согласованный закрытый канал.

22. Приёмка и передача следующему сопровождающему

Заполняйте акт фактическими результатами, а не отметками «по инструкции должно работать». Если проверка не выполнялась, так и пишите. Пункт, заблокированный окружением, остаётся открытым.

Установка и доступ

- Записаны сервер, SSH-порт, пользователь, каталог, Compose-проект, SHA и версия.
- Работают четыре сервиса; доступны вход, интерфейс и статические файлы.
- Снаружи опубликованы только согласованные порты; HTTPS и имя сертификата проверены.
- Настоящее письмо получено; вход, неверный код, выход и повторный вход проверены.
- Преподаватель видит свои данные, обычный диспетчер имеет необходимые права.
- Понижение роли проверено без суперпользователя; старый сеанс больше не даёт доступ.

Расписание

- Импорт сначала проверен без записи; ошибочный файл не изменил данные частично.
- Предварительная проверка, постановка в очередь, расчёт и корректная остановка работают.
- Проверены обе недели и сетки группы, преподавателя, аудитории.
- Пара перенесена A → B, после пересборки B → C; закрепления других пар сохранились.
- Блок английского переносится целиком; утверждение проходит проверку.
- Excel проверен по содержимому; ограничения календаря и даты действия явно переданы диспетчеру.

Сопровождение

- Ruff, mypy, миграции, SQLite, PostgreSQL и UI имеют сохранённые результаты.
- Сделана копия с проверкой состава; есть закрытая внешняя копия.
- На отдельном стенде выполнено восстановление и проверены данные/функции.
- Проверены плановый backup, контроль места, запуск после перезагрузки и порядок отката.
- Назначены владельцы сервера, домена, почты, копий и обновлений.

Шаблон передачи без значений секретов:

```
Дата / сопровождающий / принимающий:
Сервер, ОС, архитектура, ресурсы, SSH-порт:
Каталог / Compose-проект / источник кода:
Ветка / полный SHA / версия образов:
URL / DNS / ответственный за TLS:
Где защищённо хранятся доступы:
Почта и контрольный вход:
Последние отчёты SQLite, PostgreSQL, UI:
Копия: дата, имя, внешнее хранилище:
Пробное восстановление: дата, стенд, результат:
Действующий семестр и ответственный диспетчер:
Известные ограничения и следующий шаг:
Контакты владельца и администратора:
```

При смене студента владелец пересматривает доступы и API-токены. Общий root-пароль, секрет почты и резервный ZIP не являются приложениями к публичному акту передачи.

23. Шпаргалка путей и команд

Объект	Где искать
Код и основной Compose	`/opt/raspisanie`
Настройки	`/opt/raspisanie/.env`
Внешний прокси	`/opt/raspisanie/deploy/Caddyfile`
Данные PostgreSQL	Именованный том `<проект>_pgdata`; путь подтверждать inspect
TLS Caddy	Том `<проект>_caddy_data`, внутри Caddy `/data`

Объект	Где искать
Полный backup-скрипт	`/usr/local/sbin/schedule-backup.py` после установки
Копии	`/var/backups/raspisanie` после настройки
Расписание копий	`/etc/cron.d/raspisanie-backup`
Тестовая конфигурация	`docs/deployment/compose.test.yml`
Отчёты	`output/tests-sqlite.xml`, `output/tests-postgres.xml`
Согласованные требования	`docs/OPEN_QUESTIONS.md`
Оставшиеся ошибки	`docs/CODE_AUDIT_2026-11-10.md`

Частые команды из каталога рабочего проекта:

```
docker compose ps -a
docker compose logs --tail=100 worker
docker compose start
docker compose stop
docker compose exec web python manage.py check
docker compose exec web python manage.py showmigrations timetable
```

Не выполняйте `stop` и `start` подряд, если хотите оставить сервис выключенным. Не начинайте диагностику с удаления данных. Готовность подтверждают работающие функции, проверенные копии и понятный следующему сопровождающему паспорт.

24. Источники и границы документа

Команды приложения сверены с Dockerfile, Compose, entry point, моделями, командами Django, CI и документацией репозитория версии 0.14.0. Общие операции проверены по официальным источникам на дату подготовки:

- Установка Docker Engine на Ubuntu.
- Приоритеты переменных Compose.
- Изоляция проектов Compose.
- Caddy: автоматический HTTPS.
- PostgreSQL 16: pg_dump.
- PostgreSQL 16: pg_restore.

Структура и уровень подробности взяты из предоставленного `equipment-bot-deployment-kit.zip`. Секреты и команды настройки Telegram/nginx из него не переносились. После выбора заказчиком того же IP сведения о прежнем nginx добавлены как исторический контекст, требующий проверки. Обновляйте паспорт, проверки и это руководство вместе с изменениями сервиса. Историческая успешная проверка не заменяет проверку нового релиза.